

NumPy

(CC-BY: Maria Skeppstedt)

NumPy

An external Python library (= not included in standard Python).
Adds support for large, multi-dimensional arrays and matrices.
Has a lot of mathematical functions to operate on these arrays.

A basics for most machine learning.

```
1 import numpy as np
2
3 first_array = np.array([1, 2, 3])
4 print(first_array)
5 print(np.sum(first_array))
```

[1 2 3]

6

import numpy as np

has become a convention for how to import NumPy.

Creating a matrix

```
1 array_0 = ["a", "b", "c", "d", "e", "f"]
2 array_1 = ["g", "h", "i", "j", "k", "l"]
3 array_2 = ["m", "n", "o", "p", "q", "r"]
4 matrix = np.array([array_0, array_1, array_2])
5
6 print(matrix)
7 print("Shape:", matrix.shape)
8 print(f"Length first dimension: {matrix.shape[0]}")
9 print(f"Length second dimension: {matrix.shape[1]}")
10 print("At position [2, 5]:", matrix[2, 5])
```

```
[[ 'a' 'b' 'c' 'd' 'e' 'f']
 [ 'g' 'h' 'i' 'j' 'k' 'l']
 [ 'm' 'n' 'o' 'p' 'q' 'r']]
```

Shape: (3, 6)

Length first dimension: 3

Length second dimension: 6

At position [2, 5]: r

Sum matrix content

```
1 array_0 = [1, 2, 3, 4, 5, 6]
2 array_1 = [7, 8, 9, 10, 11, 12]
3 array_2 = [13, 14, 15, 16, 17, 18]
4 matrix = np.array([array_0, array_1, array_2])
5
6 print(matrix)
7 print("Dimensions:", matrix.ndim)
8 print("Sum all elements:", np.sum(matrix))
9 print("Sum over first dimension:", np.sum(matrix, axis=0))
10 print("Sum over second dimension:", np.sum(matrix, axis=1))
```

```
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]
 [13 14 15 16 17 18]]
```

Dimensions: 2

Sum all elements: 171

Sum over first dimension: [21 24 27 30 33 36]

Sum over second dimension: [21 57 93]

Create matrices

A matrix of only zeros:

```
1 np.zeros((2, 3))
```

```
array([[0., 0., 0.],  
       [0., 0., 0.]])
```

It is possible to specify datatype:

```
1 np.ones((2, 3), dtype=int)
```

```
array([[1, 1, 1],  
       [1, 1, 1]])
```

```
1 np.array([1, 2], dtype=np.float32)
```

```
array([1., 2.], dtype=float32)
```

Index for max and min

```
1 print(np.argmax(np.array([2, 3, 8, 5, 1, 7])))
```

2

```
1 print(np.argmin(np.array([2, 3, 8, 5, 1, 7])))
```

4

Combine arrays (row_stack)

```
1 matrix_1 = np.array([[1, 2, 3], [4, 5, 6]])
2 array_1 = np.array([7, 8, 9])
3 combined = np.row_stack([matrix_1, array_1])
4
5 print(matrix_1, "\n---")
6 print(array_1, "\n---")
7 print(combined)
```

```
[[1 2 3]
 [4 5 6]]
---
[7 8 9]
---
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Combine arrays (column_stack)

```
1 matrix_1 = np.array([[1, 2, 3], [4, 5, 6]])
2 array_1 = np.array([7, 8])
3 combined = np.column_stack([matrix_1, array_1])
4 print(matrix_1, "\n---")
5 print(array_1, "\n---")
6 print(combined)
```

```
[[1 2 3]
 [4 5 6]]
---
```

```
[7 8]
---
```

```
[[1 2 3 7]
 [4 5 6 8]]
```


Slicing a matrix

```
1 array_0 = ["a", "b", "c", "d", "e", "f"]
2 array_1 = ["g", "h", "i", "j", "k", "l"]
3 array_2 = ["m", "n", "o", "p", "q", "r"]
4 matrix = np.array([array_0, array_1, array_2])
5 print(matrix, "\n-----")
6
7 #All rows, the second column
8 print(matrix[:,2], "\n-----")
9
10 # Second row, all columns
11 print(matrix[2,:])
```

```
['a' 'b' 'c' 'd' 'e' 'f']
['g' 'h' 'i' 'j' 'k' 'l']
['m' 'n' 'o' 'p' 'q' 'r']]
```

```
['c' 'i' 'o']
```

```
['m' 'n' 'o' 'p' 'q' 'r']
```

Mathematical operations

```
1 np.array([1, 2, 3]) * np.array([1, 2, 3])
```

```
array([1, 4, 9])
```

```
1 np.array([1, 2, 3]) + np.array([1, 2, 3])
```

```
array([2, 4, 6])
```

Note the difference:

```
1 print([1, 2, 3] + [1, 2, 3])
```

```
[1, 2, 3, 1, 2, 3]
```

```
1 np.concatenate([np.array([1, 2, 3]), np.array([1, 2, 3])])
```

```
array([1, 2, 3, 1, 2, 3])
```

More mathematical operations

```
1 np.dot(np.array([1, 2, 3]), np.array([1, 2, 3]))
```

14

$(1*1 + 2*2 + 3*3)$

What is the dot product useful for?

For computing cosine similarity between vectors.

Also: an efficient way to multiply and combine data.

Let's say we have gathered more detailed data for determining if a creature is dangerous. It depends on four features of the creature: hunger_level, strength, cleverness, evilness.

```
1 # hunger_level, strength, cleverness, evilness
2 dangerous_weights = np.array([0.7, 0.9, 0.2, 0.8])
3
4 trollas_x = np.array([0.1, 0.7, 0.05, 0.9])
5 bilbos_x = np.array([0.9, 0.1, 0.9, 0.05])
6 evil_wizard_x = np.array([0.1, 0.9, 0.9, 0.9])
```

We have also gathered data for determining a threshold value for when a creature is dangerous, given this combined data:

```
1 # hunger_level, strength, cleverness, evilness
2 dangerous_weights = np.array([0.7, 0.9, 0.2, 0.8])
3
4 trollas_x = np.array([0.1, 0.7, 0.05, 0.9])
5 bilbos_x = np.array([0.9, 0.1, 0.9, 0.05])
6 evil_wizard_x = np.array([0.1, 0.9, 0.9, 0.9])
7
8 def is_dangerous(w, x):
9     dangerous_level = float(np.dot(w, x))
10    return (dangerous_level > 1.3, dangerous_level)
11
12 for c_x, c_name in zip([trollas_x, bilbos_x, evil_wizard_x],\
13    ["Trolla", "Bilbo", "Evil Wizard"]):
14    print(f"Is {c_name} dangerous?",\
15    is_dangerous(dangerous_weights, c_x))
```

Is Trolla dangerous? (True, 1.43)

Is Bilbo dangerous? (False, 0.9400000000000001)

Is Evil Wizard dangerous? (True, 1.78)